DispatchX

Resilient Logistics & Inventory Management System

System Documentation — v1.0

2026

1. Introduction

DispatchX is a decentralized, offline-first web platform for managing logistics chains and warehouse inventory under conditions of constrained supply, limited connectivity, and crisis operations.

The core architectural goal is optimizing distribution of limited resources using a deterministic mathematical model — the Smart Triage Engine — instead of classical FIFO queuing.

The system is built as a Progressive Web Application (PWA), enabling operation on devices ranging from dispatcher workstations to mobile driver terminals in areas with unstable internet access.

1.1 Purpose of this Document

This document provides complete technical and user-facing documentation for the DispatchX system. It covers:

- System architecture and technology decisions
- The Smart Triage Engine algorithm in detail
- Role-based access and user workflows
- Developer setup and deployment instructions
- API and data model reference

1.2 Intended Audience

Audience	Relevance
Frontend / Backend Developers	Architecture, codebase structure, algorithm implementation, deployment
Dispatchers (Logisticians)	Dashboard usage, manifest management, global radar, fleet control
Drivers (Expeditors)	Mobile HUD, manifest navigation, SOS protocol
Hub Representatives	Order placement, SKU catalog, cut-off timer, order status

2. System Architecture

2.1 Technology Stack

Layer	Technology	Notes
Framework	React 18+ (Vite)	HMR, optimized build pipeline
State Management	React Context API	GlobalStore.jsx, AuthContext.jsx
Routing	React Router	ProtectedRoute.jsx per role
GIS / Maps	Leaflet + React-Leaflet	OpenStreetMap tiles
Offline / PWA	Service Workers + Workbox	Stale-While-Revalidate strategy
PWA Install	UsePWAInstall.js hook	Windows / Android / iOS

2.2 Offline-First Paradigm

DispatchX is designed to operate without network connectivity. The offline architecture consists of two layers:

Service Workers & Workbox

The app shell (HTML, CSS, JS bundles) is pre-cached on first load. Subsequent API requests use the Stale-While-Revalidate strategy: the cached response is served immediately while a background revalidation request updates the cache.

Custom React Hooks

- UseOffline.js — global network state listener (online/offline events). Triggers UI warnings and blocks all server-dependent data mutations when offline. Queues mutations for replay on reconnect.
- UsePWAInstall.js — manages the beforeinstallprompt lifecycle, enabling the system to be installed as a native app on Windows, Android, and iOS.

2.3 Project Structure

```
├── public/                     # Static assets, PWA manifest
│   ├── favicon.svg
│   ├── icon-192.png
│   ├── icon-512.png
│   └── icons.svg
├── dev-dist/                   # Dev SW output (Workbox)
│   ├── sw.js
│   └── workbox-2ae722a1.js
├── src/
│   ├── assets/                 # Graphics and illustrations
│   ├── components/             # Base UI components
│   │   ├── features/
│   │   │   ├── auth/
│   │   │   │   └── LoginForm.jsx
```

```
│   │   │   ├── dashboard/
│   │   │   │   ├── AddRequestForm.jsx
│   │   │   │   ├── ClientDashboard.jsx
│   │   │   │   ├── Dashboard.jsx
│   │   │   │   ├── DriverDashboard.jsx
│   │   │   │   ├── RequestCard.jsx
│   │   │   │   └── StatsGrid.jsx
│   │   │   └── map/
│   │   │       └── MapViewer.jsx
│   │   ├── Button.jsx
│   │   ├── DeliveryCard.jsx
│   │   ├── ProtectedRoute.jsx
│   │   └── StatusBadge.jsx
│   ├── constants/              # Global constants
│   │   ├── routes.js
│   │   └── statuses.js
│   ├── context/                # Global state providers
│   │   ├── AuthContext.jsx
│   │   ├── GlobalStore.jsx
│   │   ├── LanguageContext.jsx
│   │   └── NotificationContext.jsx
│   ├── data/                   # Mock data (dev/testing)
│   │   └── mockData.js
│   ├── hooks/                  # Custom React hooks
│   │   ├── UseOffline.js
│   │   └── UsePWAInstall.js
│   ├── layouts/                # Page wrappers
│   │   ├── AuthLayout.jsx
│   │   └── MainLayout.jsx
│   ├── pages/                  # Route-level page components
│   │   ├── AnalyticsPage.jsx
│   │   ├── DashboardPage.jsx
│   │   ├── InventoryPage.jsx
│   │   ├── LoginPage.jsx
│   │   ├── MapPage.jsx
│   │   └── SettingsPage.jsx
│   ├── utils/                  # Pure JS algorithms
│   │   ├── generateIcons.js
│   │   ├── priority.js         # Smart Triage Engine
│   │   └── validators.js
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── index.html                  # Entry point
├── vite.config.js
├── eslint.config.js
└── package.json
```

3. Smart Triage Engine

The Smart Triage Engine is the core algorithmic module of DispatchX, implemented in src/utils/priority.js. It generates an aggregated TotalScore for every logistics transaction. Requests with the highest score are automatically promoted to the top of the dispatcher queue.

The engine runs on every iteration: on request creation, on stock change events, and on a scheduled timer.

3.1 Score Formula

Formula: TotalScore = BaseScore + WaitTimeBonus + BurnRateOverride + DeferralBonus + SpatialBonus

3.2 Component Breakdown

BaseScore — Status-Based Initialization

Initializes priority based on the declared urgency of a request. Includes quota management to prevent abuse (e.g., users marking all requests as Critical).

Status	Points	Condition
Normal	+10	Always
Elevated	+50	Always
Critical	**+100**	Only if quotaExceeded === false

WaitTimeBonus — Starvation Prevention

A linear time function that guarantees low-priority requests are eventually fulfilled. Prevents indefinite queue starvation.

Formula: +2 points per full hour since request.timestamp (Date.now() - request.timestamp)

BurnRateOverride — Proactive Depletion Control

The most critical trigger in the system. Analyzes the ratio of current stock (currentStock) to consumption speed (burnRate, units/hour). If the Time-to-Empty drops below 2 hours, or the absolute stock level falls below 20 units, the algorithm forcibly overrides all base statuses.

Trigger condition: Time-to-Empty < 2h OR currentStock < 20 units → force +200 pts

DeferralBonus — Cooperative Game Theory Incentive

If a node previously yielded its queue position voluntarily (hasDeferred: true), the system records this and awards compensatory points on the next scoring iteration. This encourages cooperative behavior between competing supply nodes.

Condition: hasDeferred === true → +30 pts on next iteration

SpatialBonus — Route Topology Optimization

When building a vehicle manifest, the engine evaluates route topology. If a pending request lies along the movement vector of an already-dispatched truck (routeMatch === true), it receives a bonus to maximize fleet utilization.

Condition: routeMatch === true → +80 pts

4. Role-Based Access Control (RBAC)

MainLayout.jsx dynamically mounts different workspaces based on the authenticated user role from AuthContext. There are three distinct roles, each with a tailored UI and permission set.

4.1 Dispatcher (System Admin / Logistician)

Workspace: High-Density "War Room" Dashboard — full system control.

Global Radar

- Interactive Leaflet map displaying real-time digital twins of all vehicles in the fleet.
- Vehicle coordinates recalculate every 200ms along polylines for accurate ETA estimation.
- Color-coded markers reflect vehicle load status and estimated arrival time.

Fleet Management

- Manifest builder: assign requests to vehicles with drag-and-drop.
- Capacity Bar: real-time validation of max vehicle load. Prevents over-assignment.
- Manifest lock: once approved, the manifest becomes immutable to hub representatives.

Global Warehouse

- Synchronized SKU inventory table across all hubs (e.g., Lviv West Hub, Odesa Port Hub).
- Per-hub stock levels, burn rates, and availability status visible in a single view.
- Inline editing of stock corrections and manual priority overrides.

Smart Queue

- Live-updating priority queue sorted by TotalScore (highest first).
- Inline display of score breakdown per request (BaseScore, BurnRateOverride, etc.).
- One-click request approval and manifest assignment.

4.2 Driver / Expeditor

Workspace: Mobile/Tablet-first HUD — minimal cognitive load, optimized for one-hand use.

Active Manifest Navigation

- List of assigned delivery stops sorted by optimized route order.
- Large confirmation button per stop: mark as delivered / partially delivered / issue.
- Auto-advances to next stop on confirmation.

SOS Crisis Protocol

- Dedicated SOS button fires an async issue_reported_toast event to the dispatcher's UI.
- Dispatcher receives an immediate alert with vehicle ID, location, and issue type.
- Enables immediate rerouting of a reserve vehicle for cargo interception.
- All SOS events are logged with timestamp and GPS coordinates.

Offline Mode

- Full manifest available offline — loaded into cache on manifest approval.
- Delivery confirmations are queued locally and synced when connectivity is restored.

4.3 Hub Representative (Customer / Recipient)

Workspace: B2B e-Commerce Portal — order placement and status tracking.

SKU Catalog

- Browse available product nomenclature (SKUs) with current stock levels.
- Nearest warehouse hub is displayed per SKU.
- Filter by category, availability, and urgency level.

Order Placement

- Create requests with status: Normal, Elevated, or Critical (quota-gated).
- Specify quantity, preferred delivery window, and receiving hub.
- Request is submitted to the dispatcher's priority queue with an initial TotalScore.

Cut-off Timer

- Each request displays a countdown to the next dispatch window.
- Once a dispatcher approves a manifest containing the order, status transitions to Locked.
- Locked orders cannot be modified by the hub representative — prevents manifest corruption.

Order Tracking

- Real-time status updates: Pending → In Queue → Assigned → In Transit → Delivered.
- ETA displayed based on vehicle GPS position and route calculation.

5. Developer Guide

5.1 Prerequisites

Requirement	Version	Notes
Node.js	18+	LTS recommended
npm	9+	Comes with Node.js
Git	Any recent	For cloning the repository

5.2 Installation & Running

Clone the repository and install dependencies:

```
git clone <repository_url>
cd dispatchx
npm install
```

Start the development server with Hot Module Replacement:

```
npm run dev
```

Build for production (also generates Service Workers via Workbox):

```
npm run build
```

Preview the production build locally:

```
npm run preview
```

5.3 Environment Variables

Create a .env file in the project root. The following variables are supported:

```
VITE_API_BASE_URL=https://api.dispatchx.internal
VITE_MAP_TILE_URL=https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png
VITE_VEHICLE_UPDATE_INTERVAL_MS=200
VITE_BURNRATE_CRITICAL_HOURS=2
VITE_BURNRATE_CRITICAL_STOCK=20
```

5.4 Key Configuration Files

File	Purpose
src/constants/statuses.js	Defines all request status strings and valid transitions
src/constants/routes.js	All client-side route paths, grouped by role
src/data/mockData.js	Seed data for local development and offline testing

src/utils/priority.js	Smart Triage Engine — core scoring algorithm
src/utils/validators.js	Input validation helpers for forms and API payloads
src/utils/generateIcons.js	Utility for dynamic SVG icon generation
src/context/GlobalStore.jsx	Global state: requests, inventory, vehicles, manifests
src/context/AuthContext.jsx	Authentication state and role management
src/context/LanguageContext.jsx	i18n language switching state
src/context/NotificationContext.jsx	Toast / alert notification state and dispatcher

6. Core Data Models

6.1 Request Object

```
{
  id:            string,      // Unique request ID
  status:        string,      // 'normal' | 'elevated' | 'critical'
  timestamp:     number,      // Unix ms — creation time (for WaitTimeBonus)
  hubId:         string,      // Destination hub identifier
  skuId:         string,      // Product SKU reference
  quantity:      number,      // Units requested
  hasDeferred:   boolean,     // True if node voluntarily yielded queue
  quotaExceeded: boolean,     // True if critical quota for this node is hit
  routeMatch:    boolean,     // True if on active vehicle route vector
  lockedAt:      number|null, // Unix ms — manifest approval timestamp
  totalScore:    number,      // Computed by priority.js
}
```

6.2 Inventory / SKU Object

```
{
  skuId:        string,   // Unique product identifier
  name:         string,   // Human-readable product name
  hubId:        string,   // Hub where stock is held
  currentStock: number,   // Current units on hand
  burnRate:     number,   // Units consumed per hour (rolling average)
  unit:         string,   // 'kg' | 'pcs' | 'l' | etc.
}
```

6.3 Vehicle Object

```
{
  id:           string,            // Vehicle identifier
  driverId:     string,            // Assigned driver user ID
  status:       string,            // 'idle' | 'en_route' | 'delivering'
  coordinates:  [number, number],  // [lat, lng] — updated every 200ms
  routePolyline: [number,number][], // Ordered waypoints
  manifestId:   string|null,       // Active manifest reference
  maxCapacity:  number,            // Max load in kg
  currentLoad:  number,            // Current load in kg
}
```

7. System Events

DispatchX uses a lightweight internal event bus for cross-module communication. Key events:

Event	Emitter	Effect
issue_reported_toast	Driver HUD	Fires SOS alert in Dispatcher War Room
manifest_locked	Dispatcher	Sets request.lockedAt; blocks hub mutations
stock_updated	Any role	Triggers priority.js recalculation for affected SKUs
vehicle_position_update	Simulation / GPS	Updates map markers and ETA calculations
offline_detected	UseOffline hook	Disables mutation buttons; shows offline banner
online_restored	UseOffline hook	Replays queued mutations; syncs with server

8. Glossary

Term	Definition
BurnRate	Rate of stock consumption in units per hour. Used to calculate Time-to-Empty.
TotalScore	Aggregated priority score computed by the Smart Triage Engine for each request.
DeferralBonus	Compensatory score bonus for nodes that previously yielded their queue position.
SpatialBonus	Score bonus applied when a request is geographically on a dispatched vehicle's route.
BurnRateOverride	Emergency score boost (+200) triggered when stock depletion is imminent.
WaitTimeBonus	Time-linear anti-starvation bonus accrued by requests waiting in the queue.
Manifest	An approved, immutable assignment of requests to a specific vehicle for delivery.
Cut-off Timer	Countdown to the next dispatch window; freezes orders once a manifest is locked.
Digital Twin	Real-time virtual representation of a physical vehicle on the Global Radar map.
SKU	Stock Keeping Unit — unique identifier for a product type in the warehouse.
PWA	Progressive Web Application — installable web app with offline and native-like capabilities.
SOS Protocol	Driver-initiated emergency event that alerts the dispatcher for immediate rerouting.
Quota Management	System-enforced limit on Critical-status requests per node to prevent abuse.
routeMatch	Boolean flag indicating a request destination lies on an active vehicle's route vector.

— End of Document —